

Solar Energy Adoption Analysis in Australia for ARENA

- **Prepared by:** Lai Kuen Tsang, Applied Data Analysis Skill Set Course at TAFE
- **Date:** 10th April, 2026
- **Confidentiality:** Internal (Assignment)

Executive Summary

This report analyses solar installation adoption trends across Australian states using installation, population, and postcode mapping datasets. The objective was to identify growth patterns, regional differences, and per-capita adoption rates to better understand Australia's solar adoption landscape.

The analysis found that solar installations have grown significantly over time, particularly after 2017, indicating increasing adoption of renewable energy technologies. States such as New South Wales and Queensland recorded the highest number of total installations, driven largely by population size and favourable solar conditions.

When adjusting for population, per-capita adoption revealed different insights, with some smaller states demonstrating relatively higher adoption rates compared to larger states. This suggests that regional policies, incentives, and environmental factors may influence adoption beyond population size alone.

Year-on-year growth analysis also showed periods of accelerated growth, likely influenced by government incentives, declining solar installation costs, and increased awareness of renewable energy benefits.

Following 2020, the growth trend begins to decline, with growth turning negative in 2022. This downturn may be partially attributed to the COVID-19 pandemic, which likely affected installation capacity, labour availability, supply chain disruptions, and overall market activity. In 2023, the growth rate rebounds to positive territory, indicating a recovery phase as market conditions stabilised and installation activity resumed. However, more recent data indicates moderating growth, suggesting the solar market in some regions may be reaching maturity.

Key findings from this analysis include:

- Strong long-term growth in solar installations across Australia
- Queensland and New South Wales leading total installations
- Variation in per-capita adoption across states
- Evidence of market maturity in recent years
- Potential growth opportunities in lower-adoption regions

These insights provide valuable information for policy makers, energy providers, and businesses to better understand solar adoption trends and identify future opportunities for renewable energy expansion.

Overall, the analysis demonstrates that solar adoption in Australia continues to grow, with regional differences presenting both challenges and opportunities for future development.

Introduction

- **Objectives:**

The objective of this analysis is to examine solar installation trends across Australian states, identify growth patterns, and evaluate adoption rates over time. The analysis aims to assess market performance using key metrics such as total installations, capacity growth, and population-normalised adoption rates.

- **Approach:**

The analysis was conducted using historical solar installation data combined with population data. The dataset was cleaned, merged, and aggregated by state, year, and quarter. Additional features including year, quarter, and per-capita installation rates were derived to support trend analysis. Key performance indicators including total installations, quarter-on-quarter growth, and year-on-year growth were calculated to evaluate performance.

Methodology

Data Sources:

There are data sources we gather from:

```
sources = { 'CER_CAP': './data/sgu-solar-capacity-2011-to-present-and-totals.csv',
'CER_INS': './data/sgu-solar-installations-2011-to-present-and-totals.csv',
```

```
#Two extra data sources:
```

```
'ABS':
```

```
'https://www.abs.gov.au/statistics/people/population/national-
state-and-territory-population/latest-release#states-and-
territories' : Population_by_State 2011_2025.csv
```

```
'AU_map postcode to states':
```

```
'https://github.com/matthewproctor/australianpostcodes/blob/master'
```

```
}
```

Data Extraction

```
In [1]: import pandas as pd
```

Load ARENA and ABS datasets

1. Installation data from ARNEA: `df_ins = pd.read_csv(../sgu-solar-installations-2011-to-present-and-totals.csv)`
2. Capacity(kw) data from ARNEA: `df_cap = pd.read_csv(../sgu-solar-capacity-2011-to-present-and-totals.csv)`
3. Mapping data of postcode and State: `df_pc = pd.read_csv(../australian_postcodes.csv)`
4. Mapping data of state and population: `df_pop = pd.read_csv(../Population_by_State 2011_2025.csv)`

Analytical Tools

The following analytical tools and techniques were used in this analysis reports:

Tools:

- Python
- Pandas for data manipulation and aggregation
- Numpy for Numerical computation
- Matplotlib for data visualization

Analytical Techniques:

- Data cleaning and preprocessing
- Data merging and tranformation
- Derved data vaiables for periods and growth analysis
- Time-series analysis
- Per capita analysis
- Trend and growth rate analysis per quarterly and yearly

Data Collation and Preparation

```
In [ ]: df_cap = pd.read_csv(r'/Users/polly/Downloads/Applied Data Analysis Skill
df_ins = pd.read_csv(r'/Users/polly/Downloads/Applied Data Analysis Skill
df_pop = pd.read_csv(r'/Users/polly/Downloads/Applied Data Analysis Skill
df_pc = pd.read_csv(r'/Users/polly/Downloads/Applied Data Analysis Skill
```

- **Read installation file as df_ins**

```
In [2]: df_ins = pd.read_csv(r'/Users/polly/Downloads/Applied Data Analysis Skill
```

df_ins

Out [2]:

	Small Unit Installation Postcode	Historic Total Installation Quantity (2001 - 2010)	Jan 2011 - Installation Quantity	Feb 2011 - Installation Quantity	Mar 2011 - Installation Quantity	Apr 2011 - Installation Quantity	May 2011 - Installation Quantity
0	0	4	0	0	0	0	0
1	200	1	0	0	0	0	0
2	800	7	0	0	0	0	0
3	801	2	0	0	0	0	0
4	803	0	0	0	0	0	0
...	...	...	...	...	...	...	...
2806	7469	2	0	0	1	0	0
2807	7470	1	0	0	0	0	0
2808	7802	1	0	0	0	0	0
2809	8576	0	0	0	1	0	0
2810	9729	1	0	0	0	0	0

2811 rows x 184 columns

In [3]: *#remove unwanted columns*
df_ins = df_ins.drop(columns=['Historic Total Installation Quantity (2001 - 2010)'])

In [4]: *#remove space*
df_ins.columns = df_ins.columns.str.strip()

In [5]: *#rename the primary key*
df_ins.rename(columns={'Small Unit Installation Postcode': 'postcode'}, inplace=True)

In [6]: *#set postcode as string and standardize it*
df_ins['postcode'] = df_ins['postcode'].astype('string')
df_ins['postcode'] = df_ins['postcode'].str.zfill(4)

In [7]: *#unpivot the table*
df_ins = pd.melt(df_ins, id_vars='postcode', var_name='date', value_name='installation_quantity')
df_ins

Out [7]:

	postcode	date	install_qty
0	0000	Jan 2011 - Installation Quantity	0
1	0200	Jan 2011 - Installation Quantity	0
2	0800	Jan 2011 - Installation Quantity	0
3	0801	Jan 2011 - Installation Quantity	0
4	0803	Jan 2011 - Installation Quantity	0
...	...	...	...
508786	7469	Jan 2026 - Installation Quantity	0
508787	7470	Jan 2026 - Installation Quantity	0
508788	7802	Jan 2026 - Installation Quantity	0
508789	8576	Jan 2026 - Installation Quantity	0
508790	9729	Jan 2026 - Installation Quantity	0

508791 rows × 3 columns

```
In [8]: # Remove " - Installation Quantity"
df_ins['date'] = df_ins['date'].str.strip().str.replace('- Installation Q
```

```
In [9]: #remove space and empty data
df_ins['date'] = df_ins['date'].astype(str).str.strip()
```

```
In [10]: #remove empty date data
df_ins = df_ins[df_ins['date'] != '']
```

```
In [11]: #set Datetime format
df_ins['date'] = pd.to_datetime(df_ins['date'], format = '%b %Y')
```

```
In [12]: df_ins
```

Out [12]:

	postcode	date	install_qty
0	0000	2011-01-01	0
1	0200	2011-01-01	0
2	0800	2011-01-01	0
3	0801	2011-01-01	0
4	0803	2011-01-01	0
...	...	...	...
508786	7469	2026-01-01	0
508787	7470	2026-01-01	0
508788	7802	2026-01-01	0
508789	8576	2026-01-01	0
508790	9729	2026-01-01	0

508791 rows × 3 columns

```
In [13]: #get year value from date
df_ins['year'] = df_ins['date'].dt.year
```

```
In [14]: #get quarter value from date
df_ins['quarter'] = df_ins['date'].dt.to_period('Q')
```

```
In [15]: #remove postcode 0000
df_ins = df_ins[df_ins['postcode'] != '0000'].reset_index(drop=True)
```

```
In [16]: #check the duplicated records
df_ins.duplicated(subset=['postcode', 'date', 'year', 'quarter']).sum()
```

Out [16]: np.int64(0)

- Read Capacity file as df_cap

```
In [17]: df_cap = pd.read_csv(r'/Users/polly/Downloads/Applied Data Analysis Skill
df_cap
```

Out [17]:

	Small Unit Postcode	Historic Total Rated Power Output In kW (2001 - 2010)	Jan 2011 - Rated Power Output In kW	Feb 2011 - Rated Power Output In kW	Mar 2011 - Rated Power Output In kW	Apr 2011 - Rated Power Output In kW	May 2011 - Rated Power Output In kW	Jun 2011 - Rated Power Output In kW	Jul 2011 - Rated Power Output In kW
0	0	4.460	0.0	0.0	0.00	0.0	0.000	0.000	0.0
1	200	0.080	0.0	0.0	0.00	0.0	0.000	0.000	0.0
2	800	34.740	0.0	0.0	0.00	0.0	2.220	0.000	3.0
3	801	3.440	0.0	0.0	0.00	0.0	2.020	0.000	0.0
4	803	0.000	0.0	0.0	0.00	0.0	0.000	0.000	0.0
...	...	...	...	...	...	...	...	...	...
2806	7469	2.100	0.0	0.0	1.56	0.0	1.645	0.000	0.0
2807	7470	1.020	0.0	0.0	0.00	0.0	0.000	0.000	0.0
2808	7802	2.450	0.0	0.0	0.00	0.0	0.000	0.000	0.0
2809	8576	0.000	0.0	0.0	1.90	0.0	0.000	0.000	0.0
2810	9729	0.875	0.0	0.0	0.00	0.0	0.000	0.000	0.0

2811 rows × 184 columns

```
In [18]: #remove unwanted columns
df_cap = df_cap.drop(columns=['Historic Total Rated Power Output In kW (2
```

```
In [19]: #remove space
df_cap.columns = df_cap.columns.str.strip()
```

```
In [20]: #rename the primary key
df_cap.rename(columns={'Small Unit Postcode':'postcode'}, inplace = True)
```

```
In [21]: #set postcode as string and standardize it
df_cap['postcode']=df_cap['postcode'].astype('string')
df_cap['postcode']=df_cap['postcode'].str.zfill(4)
```

```
In [22]: #unpivot the table
df_cap = pd.melt(df_cap,id_vars='postcode',var_name ='date',value_name='c
```

```
In [23]: df_cap
```

Out [23]:

	postcode	date	capacity_kw
0	0000	Jan 2011 - Rated Power Output In kW	0.0
1	0200	Jan 2011 - Rated Power Output In kW	0.0
2	0800	Jan 2011 - Rated Power Output In kW	0.0
3	0801	Jan 2011 - Rated Power Output In kW	0.0
4	0803	Jan 2011 - Rated Power Output In kW	0.0
...	...	...	...
508786	7469	Jan 2026 - Rated Power Output In kW	0.0
508787	7470	Jan 2026 - Rated Power Output In kW	0.0
508788	7802	Jan 2026 - Rated Power Output In kW	0.0
508789	8576	Jan 2026 - Rated Power Output In kW	0.0
508790	9729	Jan 2026 - Rated Power Output In kW	0.0

508791 rows × 3 columns

```
In [24]: # Remove " - Rated Power Output In kW"
df_cap['date'] = df_cap['date'].str.strip().str.replace(' - Rated Power 0
```

```
In [25]: #remove space and empty data
df_cap['date'] = df_cap['date'].astype(str).str.strip()
```

```
In [26]: #remove empty date data
df_cap = df_cap[df_cap['date'] != '']
```

```
In [27]: #set Datetime format
df_cap['date'] = pd.to_datetime(df_cap['date'], format = '%b %Y')
```

```
In [28]: df_cap.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 508791 entries, 0 to 508790
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   postcode        508791 non-null  string
1   date            508791 non-null  datetime64[ns]
2   capacity_kw     508791 non-null  object
dtypes: datetime64[ns](1), object(1), string(1)
memory usage: 11.6+ MB
```

```
In [29]: #convert capacity_kw datatype to float
df_cap['capacity_kw'] = (df_cap['capacity_kw'].astype(str).str.replace(', ', ''))
df_cap['capacity_kw'] = pd.to_numeric(df_cap['capacity_kw'], errors='coer
```

```
In [30]: df_cap['capacity_kw'] = df_cap['capacity_kw'].round(2)
```

```
In [31]: #get year value from date
df_cap['year'] = df_cap['date'].dt.year
```



```
In [32]: #get quarter value from date
df_cap['quarter'] = df_cap['date'].dt.to_period('Q')
df_cap
```

```
Out[32]:
```

	postcode	date	capacity_kw	year	quarter
0	0000	2011-01-01	0.0	2011	2011Q1
1	0200	2011-01-01	0.0	2011	2011Q1
2	0800	2011-01-01	0.0	2011	2011Q1
3	0801	2011-01-01	0.0	2011	2011Q1
4	0803	2011-01-01	0.0	2011	2011Q1
...	...	...	...	...	...
508786	7469	2026-01-01	0.0	2026	2026Q1
508787	7470	2026-01-01	0.0	2026	2026Q1
508788	7802	2026-01-01	0.0	2026	2026Q1
508789	8576	2026-01-01	0.0	2026	2026Q1
508790	9729	2026-01-01	0.0	2026	2026Q1

508791 rows x 5 columns

```
In [33]: #remove postcode 0000
df_cap = df_cap[df_cap['postcode'] != '0000'].reset_index(drop=True)
```

```
In [34]: #check the duplicated records
df_cap.duplicated(subset=['postcode', 'date', 'year', 'quarter']).sum()
```

```
Out[34]: np.int64(0)
```

- Read population file as df_pop

```
In [48]: df_pop = pd.read_csv(r'/Users/polly/Downloads/Applied Data Analysis Skill
df_pop
```

Out [48]:

	Unnamed: 0	Estimated Resident Population ; Male ; New South Wales ;	Estimated Resident Population ; Male ; Victoria ;	Estimated Resident Population ; Male ; Queensland ;	Estimated Resident Population ; Male ; South Australia ;	Estimated Resident Population ; Male ; Western Australia ;
0	Unit	Persons	Persons	Persons	Persons	Persons
1	Series Type	Original	Original	Original	Original	Original
2	Data Type	STOCK_CLOSE	STOCK_CLOSE	STOCK_CLOSE	STOCK_CLOSE	STOCK_CLOSE
3	Frequency	Quarter	Quarter	Quarter	Quarter	Quarter
4	Collection Month	3	3	3	3	3
...	...	...	...	...	...	...
182	Sep-2024	4239370	3446941	2771134	933594	933594
183	Dec-2024	4247233	3457529	2782864	935638	935638
184	Mar-2025	4265592	3478839	2797243	939456	939456
185	Jun-2025	4274825	3489957	2808163	941272	941272
186	Sep-2025	4291069	3505107	2819820	944311	944311

187 rows × 28 columns

```
In [49]: #remove unwanted rows
df_pop = df_pop.iloc[9:]

In [50]: #rename date columns and reset index
df_pop = df_pop.rename(columns={'Unnamed: 0': 'date'}).reset_index(drop=True)

In [51]: df_pop
```

Out [51]:

	date	Estimated Resident Population ; Male ; New South Wales ;	Estimated Resident Population ; Male ; Victoria ;	Estimated Resident Population ; Male ; Queensland ;	Estimated Resident Population ; Male ; South Australia ;	Estimated Resident Population ; Male ; Western Australia ;	Estimated Resident Population ; Male ; Tasmania ;
0	Jun-1981	2608351	1958717	1178447	653940	657249	212565
1	Sep-1981	2616060	1964139	1189946	655136	663047	212862
2	Dec-1981	2624579	1969349	1200504	657014	667381	212935
3	Mar-1982	2634534	1975617	1210128	658840	672273	213477
4	Jun-1982	2643527	1981619	1219369	660066	676892	213679
...	...	...	...	...	...	...	...
173	Sep-2024	4239370	3446941	2771134	933594	1506853	285186
174	Dec-2024	4247233	3457529	2782864	935638	1514108	285393
175	Mar-2025	4265592	3478839	2797243	939456	1525048	285616
176	Jun-2025	4274825	3489957	2808163	941272	1531884	285671
177	Sep-2025	4291069	3505107	2819820	944311	1541187	285790

178 rows × 28 columns

```
In [52]: #clean columns
df_pop.columns = (df_pop.columns.str.strip().str.replace(';', '_').str.rstrip)
```

```
In [53]: #unpivot the table
df_pop = pd.melt(df_pop, id_vars = ['date'], var_name = 'state', value_name = 'population')
```

```
In [54]: df_pop
```

Out [54]:

	date	state	population
0	Jun-1981	estimated resident population _ male _ new s...	2608351
1	Sep-1981	estimated resident population _ male _ new s...	2616060
2	Dec-1981	estimated resident population _ male _ new s...	2624579
3	Mar-1982	estimated resident population _ male _ new s...	2634534
4	Jun-1982	estimated resident population _ male _ new s...	2643527
...	...	...	...
4801	Sep-2024	estimated resident population _ persons _ au...	27301149
4802	Dec-2024	estimated resident population _ persons _ au...	27388133
4803	Mar-2025	estimated resident population _ persons _ au...	27531443
4804	Jun-2025	estimated resident population _ persons _ au...	27613654
4805	Sep-2025	estimated resident population _ persons _ au...	27724744

4806 rows × 3 columns

```
In [55]: #set date as datetime
df_pop['date'] = pd.to_datetime(df_pop['date'], format = '%b-%Y')
df_pop
```

Out [55]:

	date	state	population
0	1981-06-01	estimated resident population _ male _ new s...	2608351
1	1981-09-01	estimated resident population _ male _ new s...	2616060
2	1981-12-01	estimated resident population _ male _ new s...	2624579
3	1982-03-01	estimated resident population _ male _ new s...	2634534
4	1982-06-01	estimated resident population _ male _ new s...	2643527
...	...	...	...
4801	2024-09-01	estimated resident population _ persons _ au...	27301149
4802	2024-12-01	estimated resident population _ persons _ au...	27388133
4803	2025-03-01	estimated resident population _ persons _ au...	27531443
4804	2025-06-01	estimated resident population _ persons _ au...	27613654
4805	2025-09-01	estimated resident population _ persons _ au...	27724744

4806 rows × 3 columns

```
In [58]: #subtract state from one column
df_pop[['est', 'gender', 'state']] = df_pop['state'].str.rsplit('_', n=2, expand=True)
df_pop
```

Out [58]:

	date	state	population	est	gender
0	1981-06-01	new south wales	2608351	estimated resident population	male
1	1981-09-01	new south wales	2616060	estimated resident population	male
2	1981-12-01	new south wales	2624579	estimated resident population	male
3	1982-03-01	new south wales	2634534	estimated resident population	male
4	1982-06-01	new south wales	2643527	estimated resident population	male
...	...	...	...	...	...
4801	2024-09-01	australia	27301149	estimated resident population	persons
4802	2024-12-01	australia	27388133	estimated resident population	persons
4803	2025-03-01	australia	27531443	estimated resident population	persons
4804	2025-06-01	australia	27613654	estimated resident population	persons
4805	2025-09-01	australia	27724744	estimated resident population	persons

4806 rows × 5 columns

```
In [59]: #clean columns
df_pop['gender'] = df_pop['gender'].str.strip()
df_pop['state'] = df_pop['state'].str.strip()
```

```
In [60]: df_pop.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4806 entries, 0 to 4805
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   date             4806 non-null   datetime64[ns]
1   state            4806 non-null   object
2   population       4806 non-null   object
3   est              4806 non-null   object
4   gender           4806 non-null   object
dtypes: datetime64[ns](1), object(4)
memory usage: 187.9+ KB
```

```
In [61]: # only keep "persons" records for the population no.
df_pop = df_pop[df_pop['gender'].str.strip()=='persons']
```

```
In [62]: # drop "australia" records
df_pop = df_pop[df_pop['state'].str.strip()!='australia']
```

In [63]: `df_pop.reset_index(drop=True)`

Out [63]:

	date	state	population	est	gender
0	1981-06-01	new south wales	5234889	estimated resident population	persons
1	1981-09-01	new south wales	5249455	estimated resident population	persons
2	1981-12-01	new south wales	5266894	estimated resident population	persons
3	1982-03-01	new south wales	5286119	estimated resident population	persons
4	1982-06-01	new south wales	5303580	estimated resident population	persons
...	...	...	...	...	...
1419	2024-09-01	australian capital territory	479769	estimated resident population	persons
1420	2024-12-01	australian capital territory	481054	estimated resident population	persons
1421	2025-03-01	australian capital territory	483324	estimated resident population	persons
1422	2025-06-01	australian capital territory	484630	estimated resident population	persons
1423	2025-09-01	australian capital territory	486231	estimated resident population	persons

1424 rows × 5 columns

In [64]: `#remove unwanted columns`
`df_pop = df_pop.drop(columns = ['est','gender'])`
`df_pop`

Out [64]:

	date	state	population
3204	1981-06-01	new south wales	5234889
3205	1981-09-01	new south wales	5249455
3206	1981-12-01	new south wales	5266894
3207	1982-03-01	new south wales	5286119
3208	1982-06-01	new south wales	5303580
...	...	...	...
4623	2024-09-01	australian capital territory	479769
4624	2024-12-01	australian capital territory	481054
4625	2025-03-01	australian capital territory	483324
4626	2025-06-01	australian capital territory	484630
4627	2025-09-01	australian capital territory	486231

1424 rows × 3 columns

In [65]: `df_pop.reset_index(drop=True)`

Out [65]:

	date	state	population
0	1981-06-01	new south wales	5234889
1	1981-09-01	new south wales	5249455
2	1981-12-01	new south wales	5266894
3	1982-03-01	new south wales	5286119
4	1982-06-01	new south wales	5303580
...	...	...	...
1419	2024-09-01	australian capital territory	479769
1420	2024-12-01	australian capital territory	481054
1421	2025-03-01	australian capital territory	483324
1422	2025-06-01	australian capital territory	484630
1423	2025-09-01	australian capital territory	486231

1424 rows × 3 columns

```
In [66]: #set correct data type
df_pop['state'] = df_pop['state'].astype('string').str.strip()
df_pop['population'] = df_pop['population'].astype(int)
```

```
In [67]: # adding state_abbr

state_abbr = {
    'new south wales' : 'NSW',
    'victoria': 'VIC' ,
```

```

    'queensland': 'QLD',
    'south australia': 'SA',
    'western australia': 'WA',
    'northern territory': 'NT',
    'australian capital territory': 'ACT',
    'tasmania': 'TAS',
    'australia': 'AU'
}

```

```
df_pop['state_abbr'] = df_pop['state'].map(state_abbr)
```

In [68]: `df_pop.reset_index(drop = True)`

Out[68]:

	date	state	population	state_abbr
0	1981-06-01	new south wales	5234889	NSW
1	1981-09-01	new south wales	5249455	NSW
2	1981-12-01	new south wales	5266894	NSW
3	1982-03-01	new south wales	5286119	NSW
4	1982-06-01	new south wales	5303580	NSW
...	...	...	...	...
1419	2024-09-01	australian capital territory	479769	ACT
1420	2024-12-01	australian capital territory	481054	ACT
1421	2025-03-01	australian capital territory	483324	ACT
1422	2025-06-01	australian capital territory	484630	ACT
1423	2025-09-01	australian capital territory	486231	ACT

1424 rows x 4 columns

In [69]: `#Use state_abbr as state`
`df_pop = df_pop.drop(columns = ['state'])`
`df_pop = df_pop.rename(columns = {'state_abbr': 'state'})`
`df_pop = df_pop.reset_index(drop = True)`

In [70]: `#add quarter & year`
`df_pop['quarter'] = df_pop['date'].dt.to_period('Q')`
`df_pop['year'] = df_pop['date'].dt.year`

In [71]: `df_pop['year'] = df_pop['year'].astype('string')`
`df_pop['state'] = df_pop['state'].astype('string')`

In [72]: `# only keep year 2011 onwards to align with installation and capacity's p`
`df_pop = df_pop[df_pop['year']>='2011'].reset_index(drop=True)`
`df_pop`

Out [72]:

	date	population	state	quarter	year
0	2011-03-01	7204737	NSW	2011Q1	2011
1	2011-06-01	7218529	NSW	2011Q2	2011
2	2011-09-01	7239528	NSW	2011Q3	2011
3	2011-12-01	7258722	NSW	2011Q4	2011
4	2012-03-01	7284982	NSW	2012Q1	2012
...	...	...	...	...	...
467	2024-09-01	479769	ACT	2024Q3	2024
468	2024-12-01	481054	ACT	2024Q4	2024
469	2025-03-01	483324	ACT	2025Q1	2025
470	2025-06-01	484630	ACT	2025Q2	2025
471	2025-09-01	486231	ACT	2025Q3	2025

472 rows × 5 columns

In [73]: *# Add period 2025Q4–2026Q1 to align with install & capacity data with ffill*

```

df_pop = df_pop.sort_values(['state', 'quarter'])

fill_all_quarters = pd.period_range(
    df_pop['quarter'].min(),
    df_ins['quarter'].max(),
    freq='Q'
)

# create state + quarter combinations
idx = pd.MultiIndex.from_product(
    [df_pop['state'].unique(), fill_all_quarters],
    names=['state', 'quarter']
)

#set state and quarter as combo-key and create index
df_pop_full = (df_pop
    .set_index(['state', 'quarter'])
    .reindex(idx)    # adding missing rows
    .reset_index()
)

#fill up the data with ffill function
df_pop_full['population'] = (
    df_pop_full.groupby('state')['population'].ffill()
)

```

```

In [74]: #cope with correct date & year value as well
df_pop_full['date'] = df_pop_full['quarter'].dt.to_timestamp()
df_pop_full['year'] = df_pop_full['quarter'].dt.year

```

In [75]: df_pop_full

Out [75]:

	state	quarter	date	population	year
0	ACT	2011Q1	2011-01-01	366737.0	2011
1	ACT	2011Q2	2011-04-01	367985.0	2011
2	ACT	2011Q3	2011-07-01	369888.0	2011
3	ACT	2011Q4	2011-10-01	372070.0	2011
4	ACT	2012Q1	2012-01-01	374853.0	2012
...	...	...	...	...	...
483	WA	2025Q1	2025-01-01	3031308.0	2025
484	WA	2025Q2	2025-04-01	3044382.0	2025
485	WA	2025Q3	2025-07-01	3061672.0	2025
486	WA	2025Q4	2025-10-01	3061672.0	2025
487	WA	2026Q1	2026-01-01	3061672.0	2026

488 rows × 5 columns

- Read postcode_state file as df_pc

```
In [76]: df_pc = pd.read_csv(r'/Users/polly/Downloads/Applied Data Analysis Skill  
df_pc
```

Out [76]:

	id	postcode	locality	state	long	lat	dc
0	230	200	ANU	ACT	149.119000	-35.277700	NaN
1	21820	200	Australian National University	ACT	149.118900	-35.277700	NaN
2	232	800	DARWIN	NT	130.836680	-12.458684	NaN
3	24049	800	DARWIN CITY	NT	130.836680	-12.458684	NaN
4	233	801	DARWIN	NT	130.836680	-12.458684	NaN
...	...	...	...	...	...	...	...
18554	11186	9013	BRISBANE	QLD	152.823141	-27.603479	CITY DC - BRISBANE
18555	11187	9015	BRISBANE	QLD	152.823141	-27.603479	CITY DC - BRISBANE
18556	11196	9464	NORTHGATE MC	QLD	153.074982	-27.397055	NaN
18557	11197	9726	GOLD COAST MC	QLD	153.412197	-28.008783	NaN
18558	23878	9999	NORTH POLE	VIC	144.956776	-37.817403	CITY DELIVERY CENTRE

18559 rows x 41 columns

```

In [77]: #set primary postcode and clean the columns
df_pc['postcode'] = df_pc['postcode'].astype('string').str.strip().str.zf
df_pc['state'] = df_pc['state'].astype('string').str.strip()

In [78]: #only keep postcode and state columns
df_pc = df_pc[['postcode', 'state']]
df_pc

```

Out [78]:

	postcode	state
0	0200	ACT
1	0200	ACT
2	0800	NT
3	0800	NT
4	0801	NT
...	...	...
18554	9013	QLD
18555	9015	QLD
18556	9464	QLD
18557	9726	QLD
18558	9999	VIC

18559 rows × 2 columns

```
In [79]: # check the duplicated records
df_pc.groupby(['postcode', 'state']).size().sort_values(ascending=False)
```

```
Out[79]:  postcode  state
0822      NT      111
4702      QLD      106
2795      NSW      103
2460      NSW       97
4871      QLD       92
...
2573      NSW        1
2569      NSW        1
5455      SA         1
2568      NSW        1
9999      VIC        1
Length: 3192, dtype: int64
```

```
In [80]: #remove the duplicated rows
df_pc = df_pc.drop_duplicates(['postcode', 'state'])
```

```
In [81]: df_pc.reset_index(drop=True)
```

Out [81]:

	postcode	state
0	0200	ACT
1	0800	NT
2	0801	NT
3	0803	NT
4	0804	NT
...	...	...
3187	9013	QLD
3188	9015	QLD
3189	9464	QLD
3190	9726	QLD
3191	9999	VIC

3192 rows × 2 columns

In [82]: *#create df to adding missing postcode & state*

```
missing_pc = pd.DataFrame(
    {'postcode': ['1848', '2552', '4062', '4063', '4237', '4276', '5175',
                  '5207', '5312', '5875', '6013', '6643', '7802', '8576',
                  ],
     'state': ['NSW', 'NSW', 'QLD', 'QLD', 'QLD', 'QLD', 'SA', 'SA',
               'SA', 'SA', 'WA', 'WA', 'TAS', 'NT', 'QLD',
               ]
    })

# join the missing postcode to df_pc
df_pc_amended = pd.concat([df_pc, missing_pc], ignore_index = True)
```

In [83]: *#convert df_pc columns to string*

```
df_pc_amended['postcode'] = df_pc_amended['postcode'].astype('string')
df_pc_amended['state'] = df_pc_amended['state'].astype('string')
```

In [84]: df_pc_amended.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3207 entries, 0 to 3206
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   postcode    3207 non-null   string
1   state       3207 non-null   string
dtypes: string(2)
memory usage: 50.2 KB
```

In [85]: *#make sure one value per state-year*

```
df_pop_clean = (df_pop_full.dropna(subset=['population']).groupby(['state
```

- Merge all dataframes to analysis

```
In [86]: #use left join to merge all dfs to prevent data loss
df_merged = (df_ins
              .merge(df_cap[['postcode','date','quarter','year','capacity_kw']],on
              .merge(df_pc_amended[['postcode','state']],on = 'postcode', how = 'le
              .merge(df_pop_clean,on = ['state','year'],how = 'left')
              )
```

```
In [87]: df_merged['population'].isnull().sum()
```

```
Out[87]: np.int64(0)
```

```
In [88]: df_merged.groupby(['state','year','quarter'])['population'].nunique()
```

```
Out[88]: state  year  quarter
ACT    2011  2011Q1      1
        2011Q2      1
        2011Q3      1
        2011Q4      1
        2012  2012Q1      1
        ..
WA      2025  2025Q1      1
        2025Q2      1
        2025Q3      1
        2025Q4      1
        2026  2026Q1      1
Name: population, Length: 488, dtype: int64
```

```
In [94]: # check which merge cause the rows growth

print("df_ins:", len(df_ins))

df1 = df_ins.merge(df_cap, on=['postcode','date','quarter','year'], how='
print("after df_cap:", len(df1))

df2 = df1.merge(df_pc_amended, on='postcode', how='left')
print("after postcode:", len(df2))

df3 = df2.merge(df_pop_full, on=['state','quarter','year'], how='left')
print("after population:", len(df3))
```

```
df_ins: 508610
after df_cap: 508610
after postcode: 508610
after population: 508610
```

```
In [90]: #check duplicates
df_pc_amended['postcode'].duplicated().sum()
```

```
Out[90]: np.int64(17)
```

```
In [91]: #remove the duplicates after groupby
df_pc_amended = (df_pc_amended.groupby('postcode', as_index=False).first(
```

```
In [92]: #merged again after remove the duplicates of df_pc
df_merged = (df_ins
              .merge(df_cap[['postcode','date','quarter','year','capacity_kw']],on
              .merge(df_pc_amended[['postcode','state']],on = 'postcode', how = 'le
```

```
.merge(df_pop_clean,on = ['state','year'],how = 'left')
)
```

In [93]: df_merged

Out [93]:

	postcode	date	install_qty	year	quarter	capacity_kw	state	populati
--	----------	------	-------------	------	---------	-------------	-------	----------

0	0200	2011-01-01	0	2011	2011Q1	0.0	ACT	368930
1	0800	2011-01-01	0	2011	2011Q1	0.0	NT	231930
2	0801	2011-01-01	0	2011	2011Q1	0.0	NT	231930
3	0803	2011-01-01	0	2011	2011Q1	0.0	NT	231930
4	0804	2011-01-01	0	2011	2011Q1	0.0	NT	231930
...	...	...	...	...	...	...	...	...
508605	7469	2026-01-01	0	2026	2026Q1	0.0	TAS	576650
508606	7470	2026-01-01	0	2026	2026Q1	0.0	TAS	576650
508607	7802	2026-01-01	0	2026	2026Q1	0.0	TAS	576650
508608	8576	2026-01-01	0	2026	2026Q1	0.0	NT	265450
508609	9729	2026-01-01	0	2026	2026Q1	0.0	QLD	5692640

508610 rows × 8 columns

In [96]: df_merged['population'] = df_merged['population'].astype(int)

In [97]: df_merged[df_merged['year'] == 2025].groupby('state')['population'].unique

Out [97]:

state	population
ACT	[485430]
NSW	[8608529]
NT	[265006]
QLD	[5681203]
SA	[1905423]
TAS	[576460]
VIC	[7089104]
WA	[3053027]

Name: population, dtype: object

Data Cleaning and Handling Samples

- Cleaning Columns

```
In [ ]: #remove unwanted columns
df_ins = df_ins.drop(columns=['Historic Total Installation Quantity (2001
df_cap = df_cap.drop(columns=['Historic Total Rated Power Output In kW (2

#remove space
df_ins.columns = df_ins.columns.str.strip()
df_cap.columns = df_cap.columns.str.strip()

#df_ins['date'] = df_ins['date'].astype(str).str.strip()
#df_ins = df_ins[df_ins['date'] != '']

#rename the primary key
df_ins.rename(columns={'Small Unit Installation Postcode':'postcode'}, in
df_cap.rename(columns={'Small Unit Postcode':'postcode'}, inplace = True)
```

- Ensuring correct data types

```
In [ ]: df_cap['date'] = df_cap['date'].str.strip().str.replace(' - Rated Power 0
df_cap['date'] = pd.to_datetime(df_cap['date'], format = 'mixed')
```

- Removed duplicate records

```
In [ ]: #check the duplicated rows and remove them
df_pop.duplicated(subset=['state','quarter','year','population']).sum()
df_pop = df_pop.drop_duplicates
df_pc = df_pc.groupby(['postcode','state'],as_index=False).first()
```

- Removed unrelative records

```
In [ ]: #remove postcode 0000
df_cap= df_cap[df_cap['postcode'] != '0000'].reset_index(drop=True)
df_ins= df_ins[df_ins['postcode'] != '0000'].reset_index(drop=True)

# only keep 2011 onward years to merge installation and capacity
df_pop = df_pop[df_pop['year']>='2011'].reset_index(drop=True)
```

- Extract the data from the columns

```
In [ ]: #get year & quarter value from date column
df_cap['year']= df_cap['date'].dt.year
df_cap['quarter']= df_cap['date'].dt.to_period('Q')

#extract the person populatioin from ABS report
df_pop[['est','gender','state']]=df_pop['state'].str.rsplit('_', n=2, exp
df_pop['gender'] =df_pop['gender'].str.strip()
df_pop['state'] =df_pop['state'].str.strip()
df_pop = df_pop[df_pop['gender'].str.strip()=='persons']
df_pop = df_pop[df_pop['state'].str.strip()!='australia'].reset_index(dro

#only keep "state" column
df_pop = df_pop.drop(columns = ['est','gender'])
df_pop['state'] = df_pop['state'].astype('string').str.strip()
```


- Handling Missing Values

```
In [ ]: #create pd df to add missing postcode & state

missing_pc = pd.DataFrame(
    {'postcode': ['1848', '2552', '4062', '4063', '4237', '4276', '5175',
                  '5207', '5312', '5875', '6013', '6643', '7802', '8576',
                  ],
     'state': ['NSW', 'NSW', 'QLD', 'QLD', 'QLD', 'QLD', 'SA', 'SA',
               'SA', 'SA', 'WA', 'WA', 'TAS', 'NT', 'QLD',
               ]
    })

# join the missing postcode to df_pc
df_pc_amended = pd.concat([df_pc, missing_pc], ignore_index = True)
```

- Unpivoting data with melt function

```
In [ ]: #unpivot the table
df_ins = pd.melt(df_ins, id_vars='postcode', var_name='date', value_name='i
df_cap = pd.melt(df_cap, id_vars='postcode', var_name='date', value_name='c
df_pop = pd.melt(df_pop, id_vars = ['date'], var_name = 'state', value_name
```

Analysis and Findings

Section 1 - Overview

Total Installation Over Time:

```
In [98]: total_install = df_merged['install_qty'].sum()
print(f'Total Installations:, {total_install:,}')
```

Total Installations:, 4,027,387

Total Capacity Installed Over Time:

```
In [99]: total_capacity = df_merged['capacity_kw'].sum()
print(f'Total Capacity(KW):, {total_capacity:,}')
```

Total Capacity(KW):, 27,823,546.32

Latest Quarter Installations:

Note: Since the dataset only includes data up to January 2026, the 2026 Q1 period is incomplete. Therefore, this report considers 2025 Q4 as the most recent complete quarter.

```
In [100]: latest_quarter = df_merged['quarter'].max()
#remove incomplete quarter 2026Q1
latest_full_quarter = latest_quarter - 1
latest_install = df_merged[df_merged['quarter'] == latest_full_quarter]['
```

```
print(f'Latest Quarter: {str(latest_full_quarter)}')
print(f'Latest Installations: {latest_install:,}')

```

Latest Quarter: 2025Q4
 Latest Installations: 70,953

Growth Rate (Quarter to Quarter):

```
In [101... df_quarter = (df_merged[df_merged['quarter']!= "2026Q1"].groupby('quarter')
df_quarter['QoQ_growth'] = df_quarter['install_qty'].pct_change() * 100
df_quarter['QoQ_growth'] = df_quarter['QoQ_growth'].round(2)
#print(df_quarter)

```

Growth Rate (Year to Year):

```
In [103... # Exclude 2026 data as it is incompletd quarter period data
df_year = (df_merged[df_merged['year'] != 2026].groupby('year')['install_
df_year['YoY_growth'] = df_year['install_qty'].pct_change() * 100
df_year['YoY_growth'] = df_year['YoY_growth'].round(2)
#print(df_year)

```

Section 2 — Adoption Trends

Solar Installations Trend (Quarterly) : Quarterly Installations Over Time

```
In [104... df_quarter = (df_merged[df_merged['quarter']!= "2026Q1"].groupby('quarter')
df_quarter['QoQ_growth'] = df_quarter['install_qty'].pct_change() * 100
df_quarter['QoQ_growth'] = df_quarter['QoQ_growth'].round(2)
#print(df_quarter)

```

```
In [105... import matplotlib.pyplot as plt
import numpy as np
import matplotlib.ticker as ticker

#calculate the quarterly installtion ove time but exclude 2026Q1 as imcom
df_quarter_ins = (df_merged[df_merged['quarter']!= "2026Q1"].groupby('qua

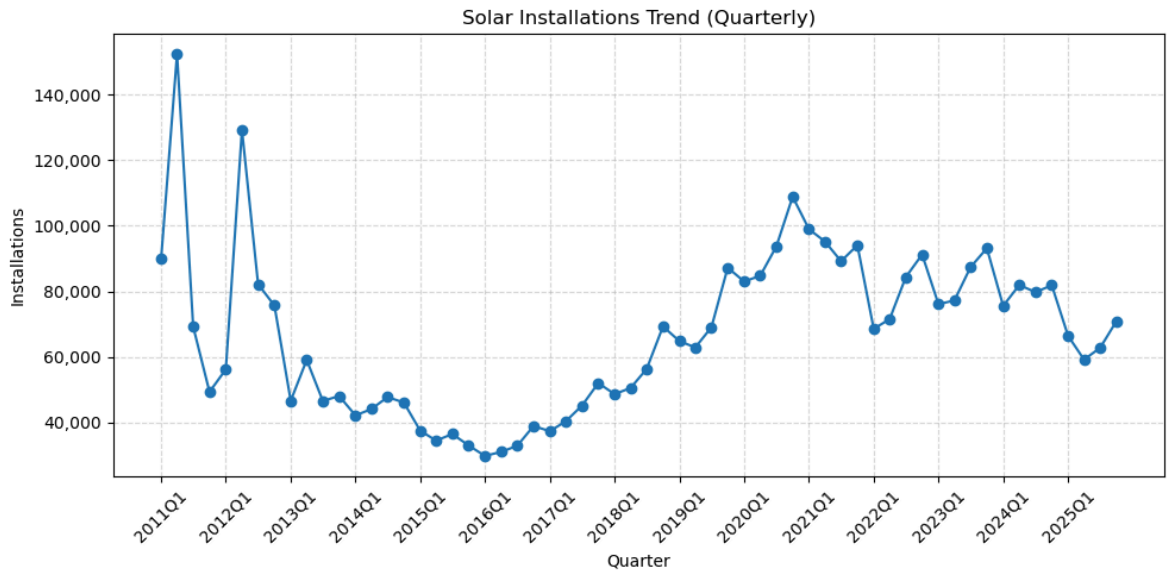
#plot the data to line chart

plt.figure(figsize=(10,5))
plt.plot(df_quarter_ins['quarter'].dt.to_timestamp(), df_quarter_ins['ins
plt.title('Solar Installations Trend (Quarterly)')
plt.xlabel('Quarter')
plt.ylabel('Installations')
plt.gca().yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

plt.grid(True, linestyle='--', alpha=0.5)
plt.xticks(
    df_quarter_ins['quarter'].dt.to_timestamp()[::4],
    df_quarter_ins['quarter'].astype(str)[::4],
    rotation=45)

plt.tight_layout()
plt.show()

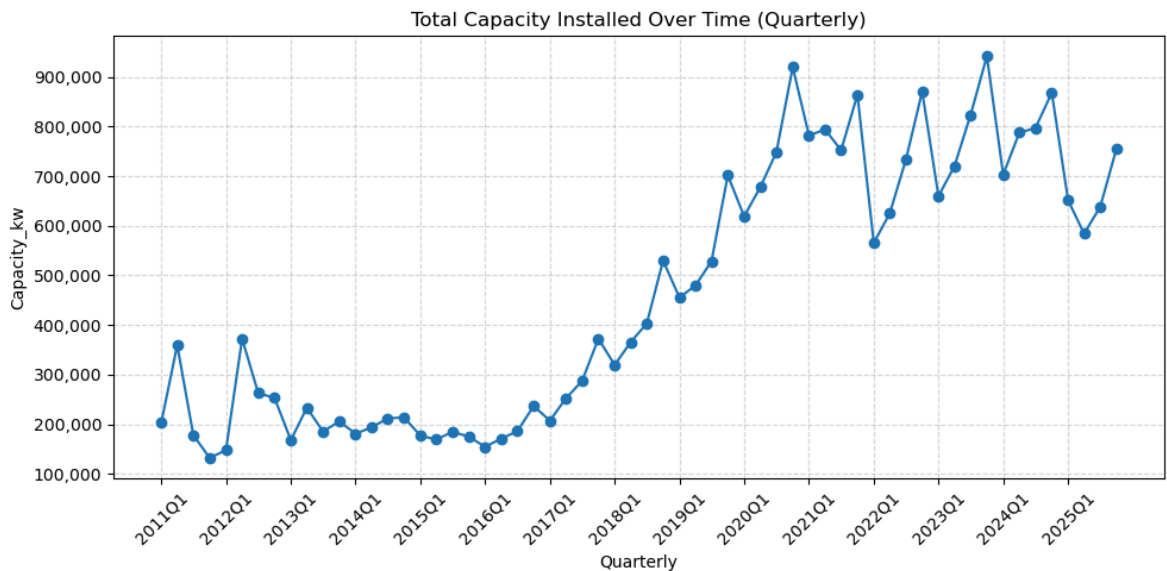
```



Capacity Growth Trend : Total Capacity Installed Over Time

```
In [106... # calculate the Total Capacity Installed Over Time
df_quarter_cap = (df_merged[df_merged['quarter']!= "2026Q1"].groupby('qua

#Plot the data to line chart
plt.figure(figsize=(10,5))
plt.plot(df_quarter_cap['quarter'].dt.to_timestamp(), df_quarter_cap['cap
plt.title('Total Capacity Installed Over Time (Quarterly)')
plt.xlabel('Quarterly')
plt.ylabel('Capacity_kw')
plt.gca().yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}'))
)
plt.grid(True, linestyle='--', alpha=0.5)
plt.xticks(
    df_quarter_cap['quarter'].dt.to_timestamp()[::4],
    df_quarter_cap['quarter'].astype(str)[::4],
    rotation=45
)
plt.tight_layout()
plt.show()
```



The insights from solar installation and capacity installed over the time:

- solar adoption increased significantly after 2017
- The growth was being accelerated during recent years.

Combine installation and capacity trend

Note: The data has been normalized for visualization purposes, as installation and capacity are measured on different scales.

```
In [107... #plot the Installation vs Capacity Collrelation View

ins = df_quarter_ins.copy()
cap = df_quarter_cap.copy()

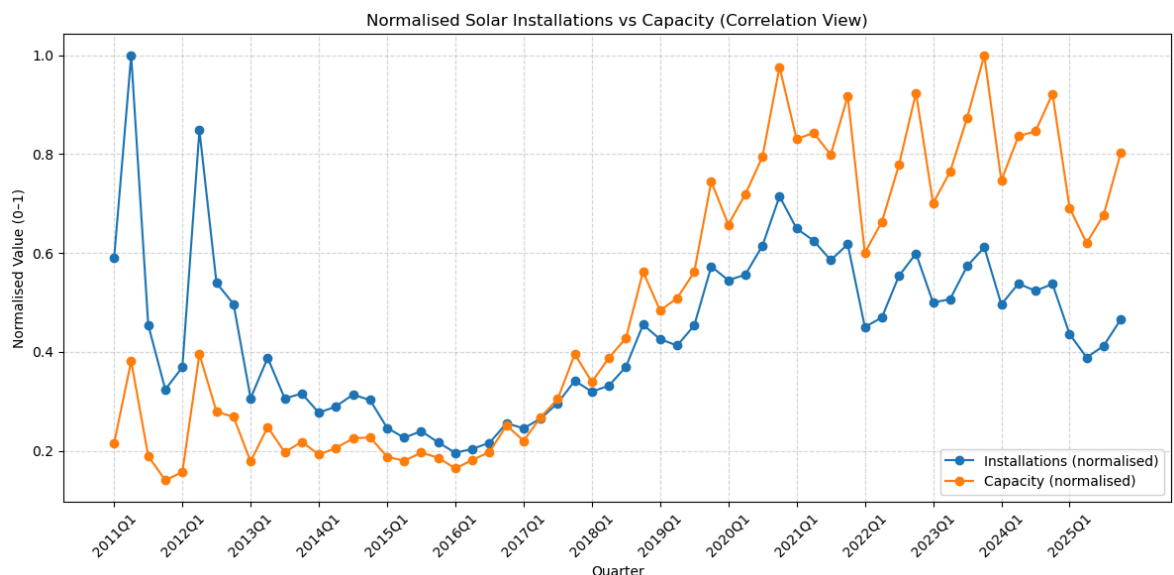
ins['install_norm'] = ins['install_qty'] / ins['install_qty'].max()
cap['capacity_norm'] = cap['capacity_kw'] / cap['capacity_kw'].max()

plt.figure(figsize=(12,6))

plt.plot(ins['quarter'].astype(str), ins['install_norm'], label='Installations (normalised)')
plt.plot(cap['quarter'].astype(str), cap['capacity_norm'], label='Capacity (normalised)')

plt.title('Normalised Solar Installations vs Capacity (Correlation View)')
plt.xlabel('Quarter')
plt.ylabel('Normalised Value (0-1)')
plt.xticks(ins['quarter'].astype(str)[::4], rotation=45)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```



- Overall Trend (Installations vs Capacity) insights:

Both solar installations and installed capacity show a strong upward trend over time, indicating continuous adoption of rooftop solar systems. Even if installation growth slows, total energy generation potential can still rise strongly. However, installation

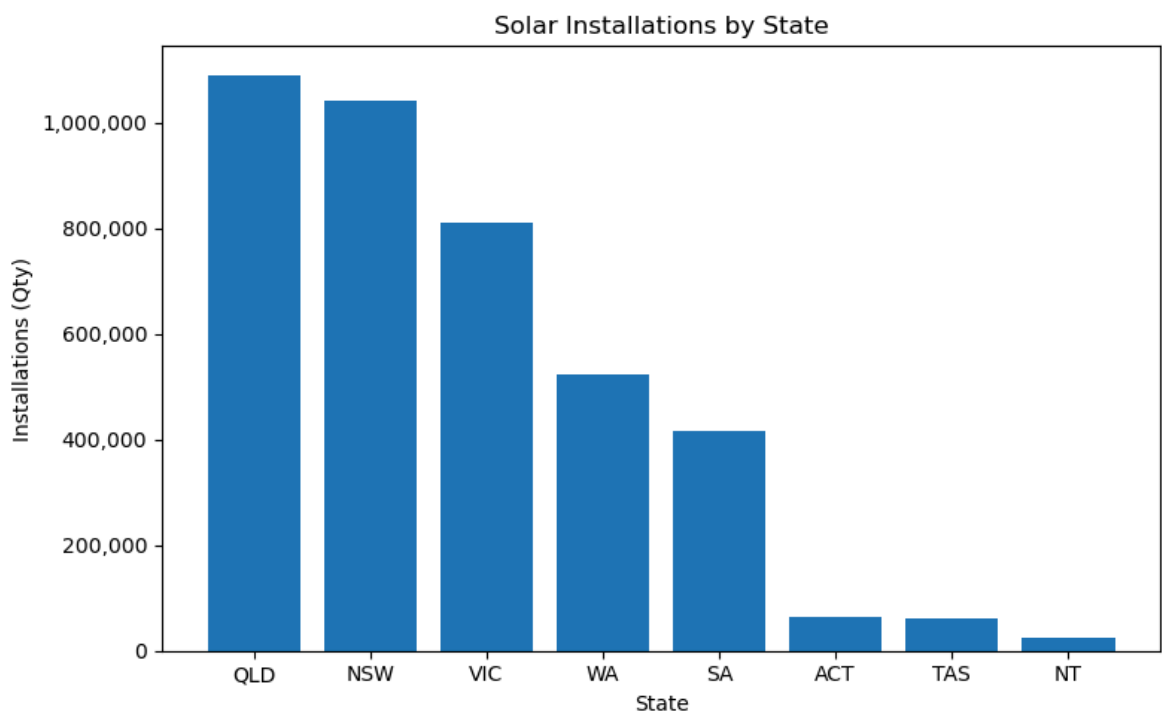
growth and capacity growth do not always move at the same speed, suggesting changes in system size over time.

Section 3 — Geographic Insights

State Comparison Dashboard :Installations by State

```
In [108... #compare the state installation
df_state = (df_merged.groupby('state')['install_qty'].sum()).sort_values(a

# plot the data to bar chart
plt.figure(figsize=(8,5))
plt.bar(df_state['state'], df_state['install_qty'])
plt.title('Solar Installations by State')
plt.xlabel('State')
plt.ylabel('Installations (Qty)')
plt.gca().yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)
plt.tight_layout()
plt.show()
```



Top 10 Solar Postcodes:

```
In [111... # group top 10 postcode with state
df_top_pc = (
    df_merged.groupby(['postcode', 'state'])['install_qty']
        .sum()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
)

# create combined label
df_top_pc['postcode_state'] = (
    df_top_pc['postcode'].astype(str) + ' (' + df_top_pc['state'] + ')'
```

```

)

# plot
plt.figure(figsize=(9,6))

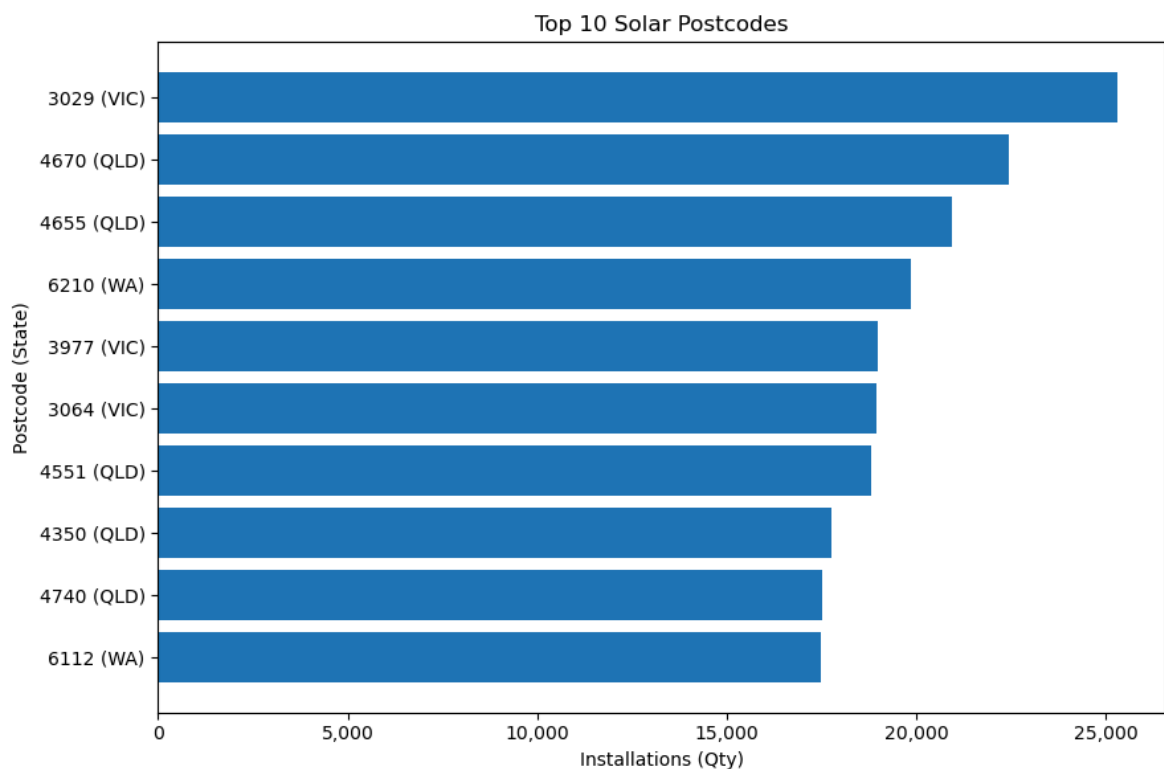
plt.barh(
    df_top_pc['postcode_state'],
    df_top_pc['install_qty']
)

plt.gca().invert_yaxis()
plt.title('Top 10 Solar Postcodes')
plt.xlabel('Installations (Qty)')
plt.ylabel('Postcode (State)')

plt.gca().xaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

plt.tight_layout()
plt.show()

```



Section 4 — Solar Adoption Rate Per Capita Analysis

- Solar Adoption Rate by State 2011-2025 (per capita)

```

In [112... import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

df_install_state = (df_merged.groupby('state')['install_qty'].sum()).reset_index()
df_pop_state = (df_merged.groupby('state')['population'].mean()).reset_index()

df_per_cap = df_install_state.merge(df_pop_state, on='state')
df_per_cap['install_per_capita'] = (df_per_cap['install_qty'] / df_per_cap['population'])

```

```

df_per_cap = df_per_cap.sort_values('install_per_capita', ascending=True)

# Plot
plt.figure(figsize=(10,6))

bars = plt.barh(df_per_cap['state'],df_per_cap['install_per_capita'])

plt.title('Solar Adoption Rate by State (2011-2025)')
plt.xlabel('Adoption Rate (%)')
plt.ylabel('State')

# Format as percentage
plt.gca().xaxis.set_major_formatter(ticker.PercentFormatter(1))

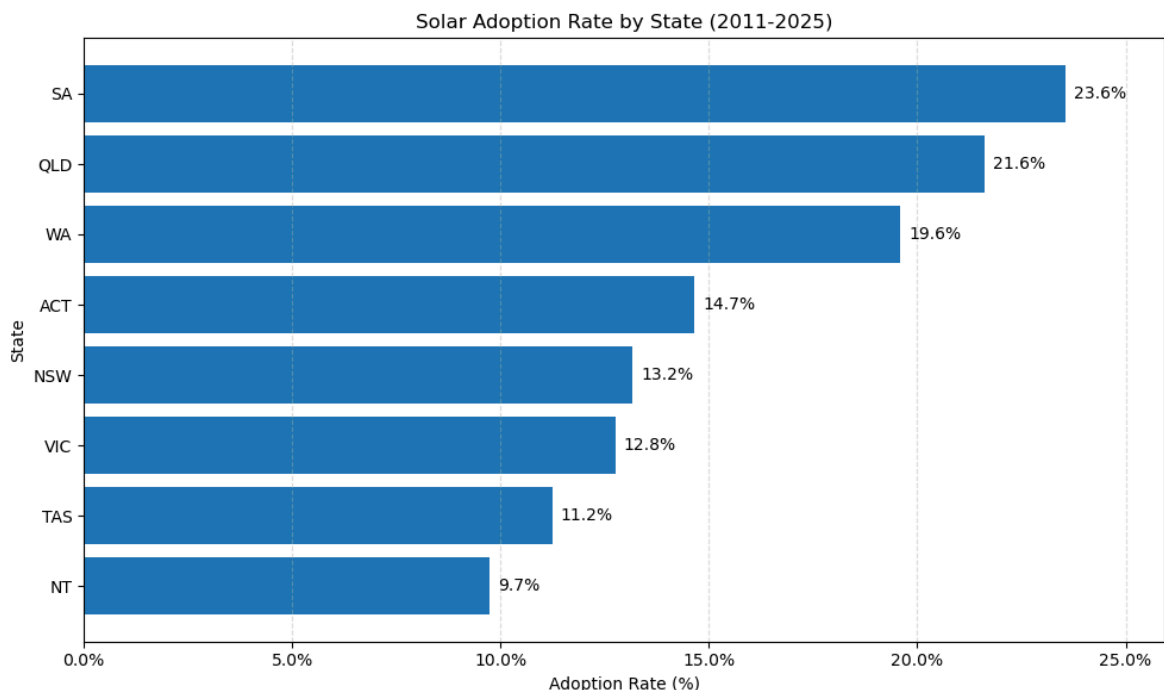
# Add labels
for bar in bars:
    width = bar.get_width()
    plt.text(
        width + 0.002,
        bar.get_y() + bar.get_height()/2,
        f'{width:.1%}',
        va='center'
    )

plt.xlim(0, df_per_cap['install_per_capita'].max()*1.1)

plt.grid(axis='x', linestyle='--', alpha=0.4)

plt.tight_layout()
plt.show()

```



- Solar Adoption Rate by State Insights:

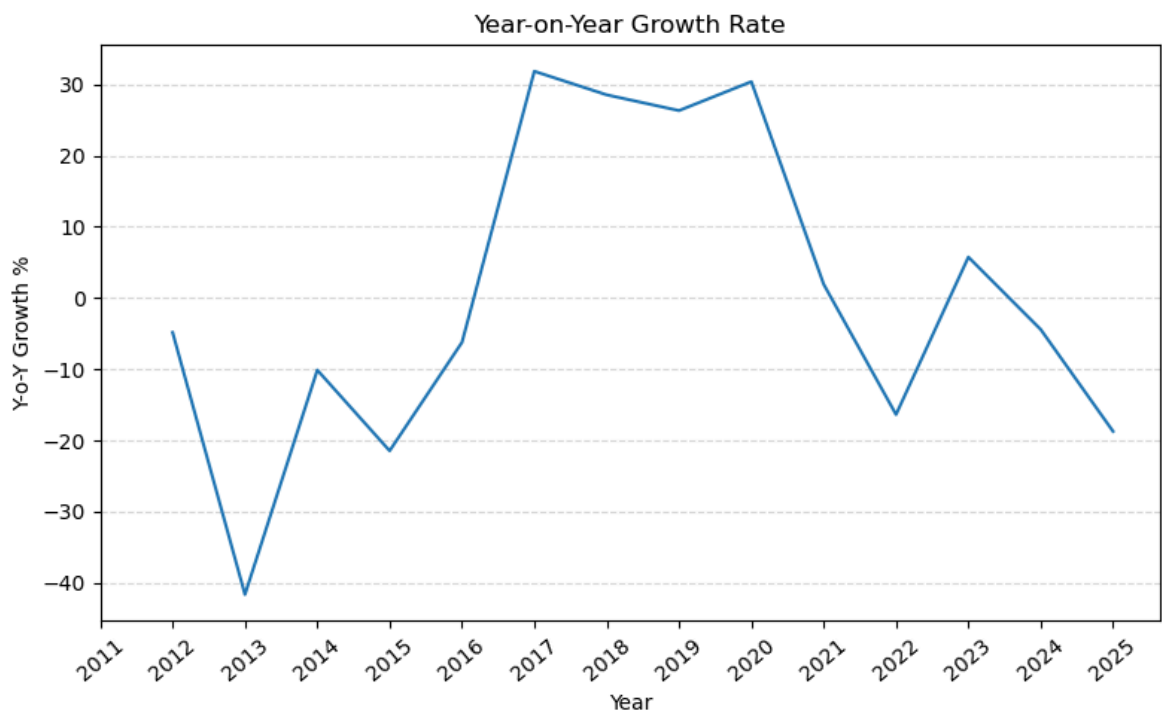
South Australia leads solar adoption rate (installation per capita) at 23.6%, followed by Queensland and Western Australia, while Tasmania and Northern Territory show the lowest uptake, highlighting strong regional differences influenced by sunlight, housing type, and policy incentives.

Section 5 — Year-on-Year Growth

```
In [113... #Year on Year Growth
df_year = (df_merged.groupby('year')['install_qty'].sum().reset_index())
df_year = df_year[df_year['year'] < 2026] # exclude 2026 incompleted quar
df_year['growth'] = df_year['install_qty'].pct_change() * 100

# plot the year-on-year trend

plt.figure(figsize=(8,5))
plt.plot(df_year['year'], df_year['growth'])
plt.grid(True, axis = 'y',linestyle='--', alpha=0.5)
plt.title('Year-on-Year Growth Rate')
plt.xlabel('Year')
plt.ylabel('Y-o-Y Growth %')
plt.xticks(df_year['year'],rotation = 40)
plt.tight_layout()
plt.show()
```



- Year-On-Year Chart Insights:

Between 2016 and 2017, there is a strong spike in growth, indicating rapid adoption of solar installations. From 2017 to 2020, growth remains consistently positive, suggesting strong market expansion during this period. From 2020 onwards, there is a noticeable decline, which may be partly influenced by the COVID-19 pandemic, affecting supply chains, workforce availability, and installation activities. Growth becomes negative in 2022, before recovering in 2023, indicating a temporary slowdown followed by market recovery. However, from 2023 to 2025, growth declines again with policy changed, some states changed direct rebate policy which contribute the growth drop trend, such as QLD, VIC, SA & TAS. However, year-on-year growth demonstrates a consistent long-term increase in solar adoption, indicating sustained market expansion.

Discussion

The analysis shows clear differences in rooftop solar adoption across Australian states (2011–2025), driven by geography, housing type, and policy support.

South Australia leads significantly with 23.6% adoption, reflecting strong incentives and high energy costs. Queensland (21.6%) and Western Australia (19.6%) also show strong uptake due to favourable sunlight conditions and higher rates of detached housing.

New South Wales (13.2%) and Victoria (12.8%) show moderate adoption despite large populations, likely due to urban density and more apartment living. Tasmania (11.2%) and Northern Territory (9.7%) record the lowest adoption, influenced by environmental and infrastructure limitations.

Overall, solar adoption varies widely across states, with stronger performance in sunnier and less urbanised regions.

Recommendations

1. Target NSW and VIC for adoption growth

Since NSW and VIC represent large populations but relatively lower adoption rates, we can make them key growth areas. Improve uptake through higher rebates, faster approvals, and support for apartment solar solutions. My recommended Policy interventions such as:

- Increased rebates for rooftop solar
- Streamlined approval processes for installations
- Incentives for apartment solar solutions

Those processes could significantly improve uptake.

2. Maintain and expand incentives in high-performing states (SA, QLD, WA)

South Australia, Queensland, and Western Australia already show strong adoption. However, continued support is important to:

- Sustain momentum
- Encourage battery storage adoption
- Prevent market saturation slowdown

We can focus shift from adoption to energy storage and system optimisation.

3. Develop tailored strategies for low-adoption regions (TAS, NT)

Tasmania and Northern Territory require region-specific strategies, such as:

- Community solar projects (shared solar farms)
- Government-supported installations
- Off-grid or hybrid energy systems in remote areas

These regions may not scale through rooftop solar alone due to environmental and demographic constraints.

4. Promote apartment and urban solar solutions

Lower adoption in NSW and VIC highlights the need for:

- Shared solar schemes (virtual net metering)
- Building-integrated solar systems
- Community energy models

This addresses structural barriers in high-density urban environments.

5. Enhance data-driven monitoring of solar adoption

To improve future decision-making:

1. Track adoption at quarterly/state level consistently
2. Standardise population adjustments (per capita metrics)
3. Incorporate weather, income, and housing data for deeper insights

This will enable more precise policy targeting and forecasting.

References

- Clean Energy Regulator, Australian Government (2026), Quarterly Carbon Market Report September Quarter 2022, <https://cer.gov.au/markets/reports-and-data/quarterly-carbon-market-reports/quarterly-carbon-market-report-september-quarter-2022/small-scale-technology-certificates-stcs>
- Department of Climate Change, Energy, the Environment and Water (2024), Solar rebates and incentives, Australian Government, viewed 18 April 2026, <https://www.energy.gov.au>

Appendices

Extra data source:

1. <https://www.abs.gov.au/statistics/people/population/national-state-and-territory-population/latest-release#states-and-territories>, Population_by_State 2011_2025.csv

2. https://github.com/matthewproctor/australianpostcodes/blob/master/australian_postcodes.csv

– End

–